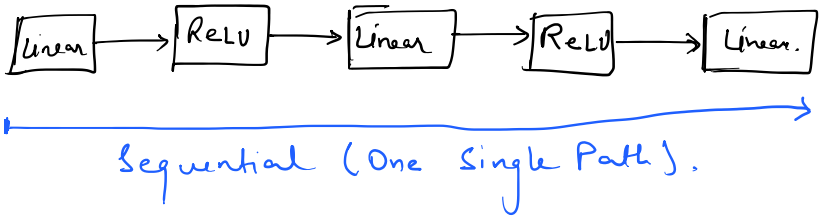
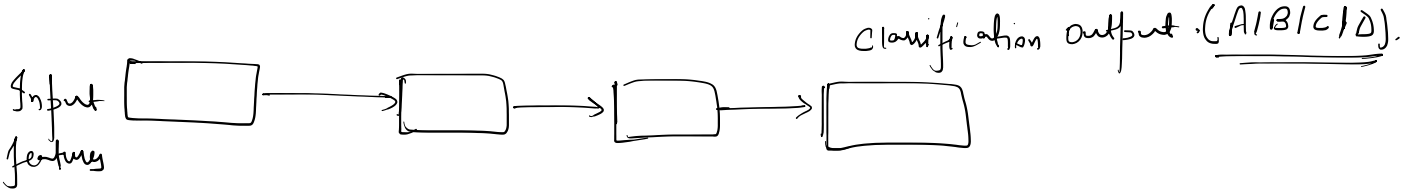
PyTorch 2: Training NNs in Practice

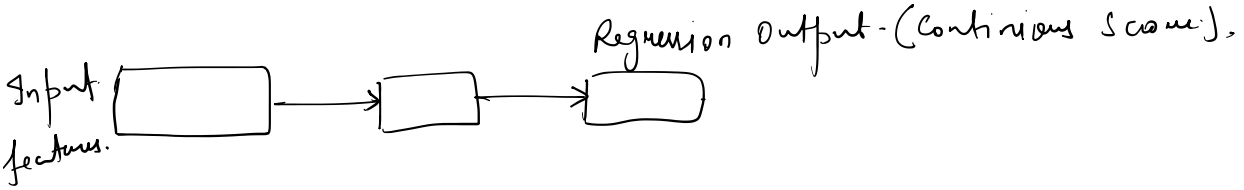
Tuesday, 6 October 2026 1:24 AM

* PyTorch Sequential

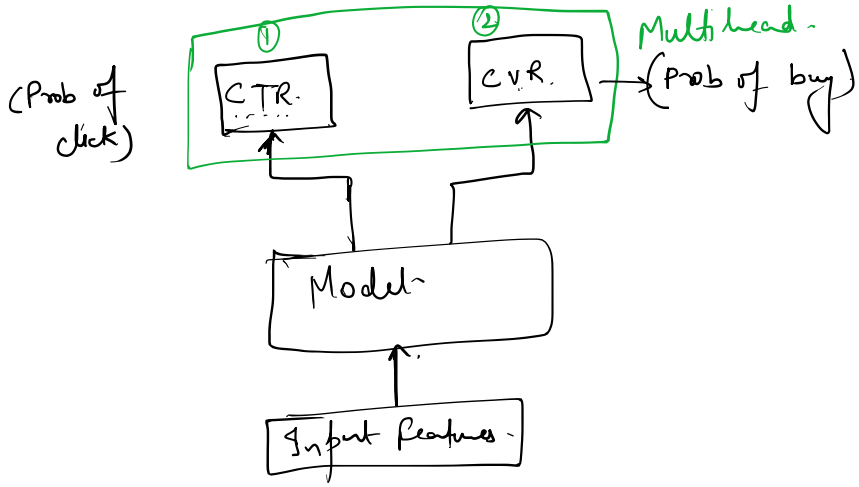


Healthify me,

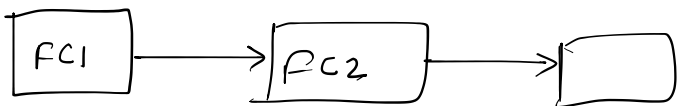




Goal: To learn how to create non-sequential model.



Branch 1 :



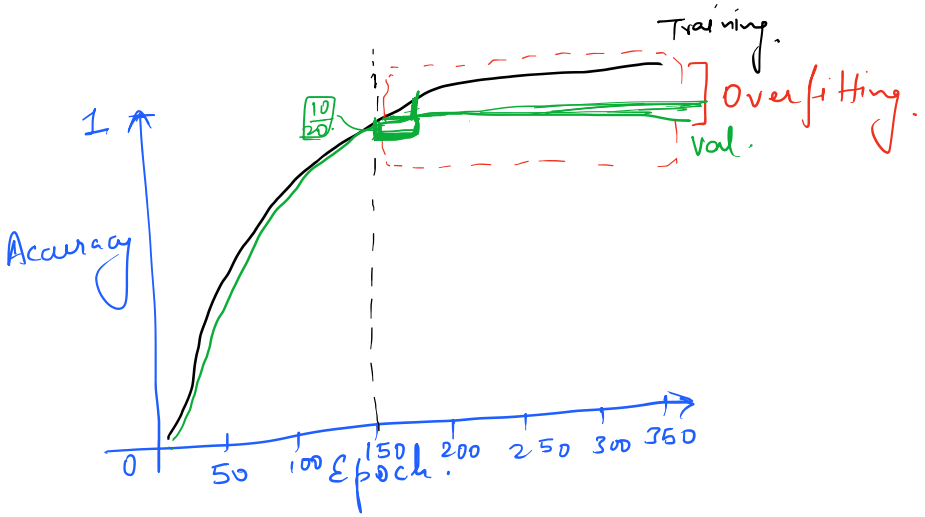
1x16 16x4 4x4.

$\frac{7}{11} = \boxed{7/11}$ → accuracy.

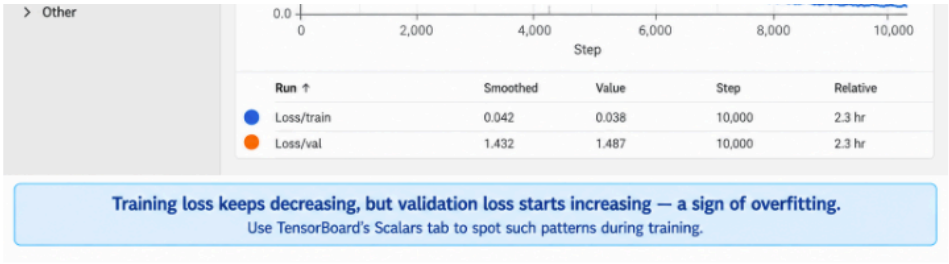
Softmax → ①

Cross Entropy Loss (x, y) = Neg log likelihood (Softmax (x) , y).
② ↑

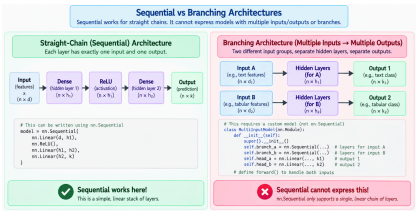
* Monitoring and Controlling Training.

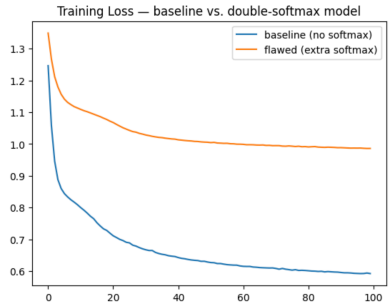






Training loss keeps decreasing, but validation loss starts increasing — a sign of overfitting.
Use TensorBoard's Scalars tab to spot such patterns during training.


Sequential vs Branching Architectures
Sequential works for straight chains. It cannot express models with multiple inputs/outputs or branches.
Straight-Chain (Sequential) Architecture
Each layer has exactly one input and one output.
Input
Dense
ReLU
Dense
Output
This can be written using nn.Sequential
model = nn.Sequential(
nn.Linear(d, h1),
nn.ReLU(),
nn.Linear(h1, h2),
nn.Linear(h2, k)
)
Sequential works here!
This is a simple, linear stack of layers.
Branching Architecture (Multiple Inputs → Multiple Outputs)
Two different input groups, separate hidden layers, separate outputs.
Input A
Hidden Layers
(for A)
Output 1
Input B
Hidden Layers
(for B)
Output 2
This requires a custom model (not nn.Sequential)
class MultiInputModel(nn.Module):
def __init__(self):
super().__init__()
self.branch_a = nn.Sequential(...) # layers for input A
self.branch_b = nn.Sequential(...) # layers for input B
self.head_1 = nn.Linear(..., k1) # output 1
self.head_2 = nn.Linear(..., k2) # output 2
define forward() to handle both inputs
Sequential cannot express this!
nn.Sequential only supports a single, linear chain of layers.


Training Loss — baseline vs. double-softmax model
baseline (no softmax)
flawed (extra softmax)
1.3
1.2
1.1
1.0
0.9
0.8
0.7
0.6
0
20
40
60
80
100